

Deep Q-Network Optimization: From Theoretical Foundations to Advanced Control in CartPole-v1

Author: Mohammadmahdi Bababeyk

StudentID: 4041419005

1. Theoretical Prologue: Understanding Model Capacity and VC Dimension

In the development of robust Reinforcement Learning (RL) agents, we must first rigorously evaluate the capacity of our function approximators. A model's ability to learn is fundamentally bounded by its complexity, often measured through the Vapnik-Chervonenkis (VC) Dimension. Before applying high-capacity neural networks to complex control problems, we analyze the limitations of simpler models to understand the necessity of non-linear architectures.

Consider the linear decision function $f(x) = w \cdot x$. This model has a VC Dimension of exactly 1. While it can "shatter" a single point by shifting the boundary to assign either a 0 or 1 label, it fails to shatter a set of two points. For any two points x_1, x_2 , this function cannot produce the label sequence (1, 0). Because the function is inherently monotonic, if $f(x_1) > 0$ (assigning label 1), then $f(x_2)$ must also be greater than 0, forcing a label 1. This inability to represent non-monotonic decision boundaries is the primary reason linear models are insufficient for the "wiggles" required to map complex state-action spaces. To achieve higher capacity, we must transition to deep neural networks utilizing non-linear activation functions like ReLU, which allow the model to approximate the intricate, non-monotonic Q-value surfaces necessary for environments like CartPole-v1.

2. The Deep Reinforcement Learning Paradigm and the DQN Breakthrough

The transition from tabular Q-learning to Deep RL was necessitated by the "curse of dimensionality." In the CartPole-v1 environment, the state space is a continuous 4D vector; a tabular approach would require infinite memory to store every real-valued coordinate. Function approximation via neural networks allows for generalization across this continuous space, yet naively merging Q-learning with deep learning introduces three "catastrophic" instabilities:

1. **Non-stationary Target Distributions:** The Q-learning target relies on the current network weights, meaning the "ground truth" changes with every update, leading to a volatile "moving target" problem.
2. **Correlated Data:** Sequential observations in RL are temporally linked, violating the independent and identically distributed (i.i.d.) assumption essential for stable stochastic gradient descent.
3. **Catastrophic Forgetting:** Without a mechanism to preserve past experiences, an agent can overfit to recent trajectories, losing critical knowledge of the global state space.

The 2015 DQN framework by Mnih et al. introduced Experience Replay and Target Networks as the "connective tissue" to solve these issues. Experience Replay enhances sample efficiency by

reusing data, while the Target Network provides a stable bootstrap objective. These innovations allow us to treat RL as a supervised learning problem, approximating the i.i.d. assumption and stabilizing the gradient descent process.

3. Architectural Framework: Implementing the Q-Function Approximator

In continuous environments, the architecture of the Q-network is a strategic lever for balancing bias and variance. For CartPole-v1, we implement a multi-layer perceptron designed to perform high-fidelity regression on action values.

Q-Network Architecture Summary

Layer	Specifications	Rationale
Input	4D State Vector	Maps cart position, velocity, pole angle, and angular velocity.
Hidden 1	128 Units, ReLU	Initiates non-linear feature extraction from the raw state.
Hidden 2	128 Units, ReLU	Increases model capacity to approximate complex Q-value surfaces.
Output	2 Units (Raw Logits)	Predicts scalar Q-values for the two discrete action choices.

Notably, we avoid a softmax activation on the output. DQN performs regression to predict cumulative rewards rather than classification to predict probabilities. Action selection is governed by an ϵ -greedy strategy, where ϵ decays from 1.0 to 0.01. This ensures a robust exploration-exploitation trade-off, allowing the agent to sample the environment extensively before specializing in high-value trajectories.

4. Advanced Enhancements: Beyond the Standard DQN

To mitigate common biases and ensure stable convergence, our implementation incorporates several critical optimizations beyond the vanilla DQN architecture.

Standard DQN is prone to overestimation bias because it uses the same weights to both select and evaluate an action. We implement Double DQN (DDQN) logic to decouple these processes: the policy network selects the optimal action, while the target network provides the value estimation. This results in more conservative and accurate Q-value updates.

Furthermore, we transition from Mean Squared Error (MSE) to Huber Loss (SmoothL1Loss). In early training, Temporal Difference (TD) errors can be extremely large. MSE's squared term would lead to "exploding" gradients; Huber Loss provides a linear tail for large errors, ensuring stable, robust gradient updates. Finally, we utilize Soft Updates (Polyak Averaging) with τ . Instead of a periodic "hard" copy of weights, the target network parameters slowly track the policy

network. This ensures the target distribution remains quasi-stationary, significantly smoothing the training trajectory.

5. Performance Analysis and Experimental Results

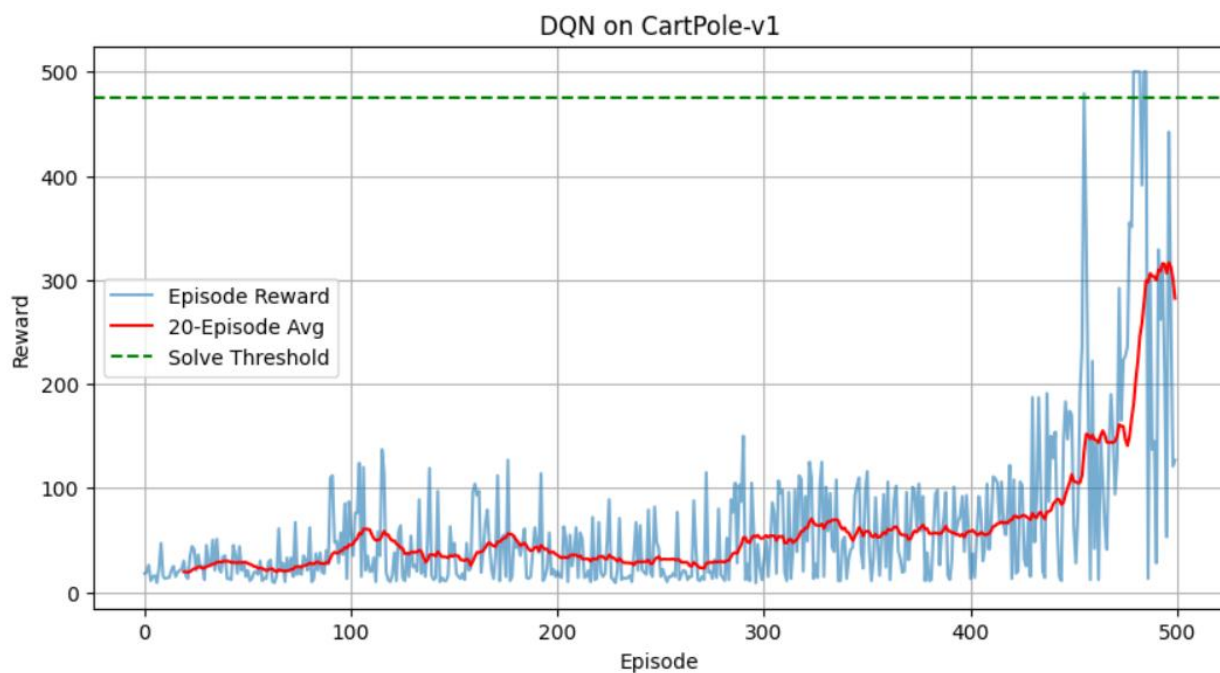
Visualizing training metrics is essential for diagnosing the agent's convergence and the stability of the policy. The following table tracks the agent's progress over 500 episodes using the optimized DDQN framework.

Training Progression Summary

Episode Avg Reward (Last 20) Exploration Rate ()

0	18.00	0.995
100	45.20	0.603
200	35.50	0.365
300	52.55	0.221
400	58.35	0.134
480	203.55	0.090

The figure below, "DQN on CartPole-v1," displays the reward trajectory across the full experimental horizon.



Analysis of the plot reveals that while the agent shows a significant upward trajectory and occasional episodes reaching the 500-step limit, it did not consistently solve the environment within the 500-episode window. The "20-Episode Avg" (red line) shows clear improvement, rising from a baseline of ~20 to a peak average of ~203.55. However, the agent remained below the green dashed "Solve Threshold" of 475. This performance gap suggests that while the Double DQN and Huber Loss optimizations were effective, a longer training horizon or more aggressive -decay tuning may be required to reach full environment mastery.

6. Conclusion and Strategic Takeaways

The synergy between theoretical model capacity and practical algorithmic stability is the hallmark of modern Deep RL. Our analysis of the VC Dimension established the need for non-linear function approximators, while the implementation of Double DQN and robust loss functions provided the necessary stability to train those high-capacity models in a continuous 4D state space.

Strategic Takeaways for Stable Deep RL

- **Target Stability:** Employing separate target networks with Polyak averaging is mandatory to mitigate the instabilities of non-stationary target distributions.
- **Robust Optimization:** Huber Loss () is superior to MSE in RL contexts, as it prevents gradient explosion during early-stage training when TD-errors are high.
- **Sample Decorrelation:** Experience Replay is the primary tool for breaking temporal correlations and maximizing sample efficiency through data reuse.
- **Environment Standardization:** Transitioning to modern frameworks like Gymnasium is essential for maintaining reproducibility and compatibility with current NumPy and PyTorch standards.

While the agent reached significant performance milestones, the failure to reach the 475-reward threshold highlights the sensitivity of Deep RL to hyperparameters. These techniques remain highly scalable to more complex Gymnasium environments provided there is sufficient tuning of the training horizon.

7. References

- Farama Foundation. (n.d.). *Gymnasium documentation: Introduction and migration guide*. Retrieved from https://gymnasium.farama.org/introduction/migration_guide/.
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533